# CS34020/CSM4220 Workshop 3

# Object Detection/Recognition

Theme:
Security

**Purpose and theme:** in this workshop we will try out some methods for detecting objects and then recognising them if we re-encounter them. The tasks in this workshop are loosely around the theme of *security* but could be applied in other domains.

## Part 1: Finding Faces with Viola-Jones

Face detection and recognition is a common requirement in security applications. Let's find some faces using the Viola-Jones algorithm. Start a new file using the following outline which gives you two functions to do most of the work. **Note you will need to fix indentation**:

```
# ADD CODE HERE: import opencv

# our pre-trained classifiers
cascade=cv.CascadeClassifier("haarcascade_frontalface_default.xml")
# function 1: detects faces and returns their locations
def detect(img, cascade):
     rects = cascade.detectMultiScale(img, scaleFactor=1.3,
                 minNeighbors=4, minSize=(30, 30),
                 flags=cv.CASCADE_SCALE_IMAGE)
     if len(rects) == 0:
        return []
     rects[:,2:] += rects[:,:2]
     return rects

# function 2: draws the bounding boxes on the image def
draw_rects(img, rects, color):
     for x1, y1, x2, y2 in rects:
        cv.rectangle(img, (x1, y1), (x2, y2), color, 2)

# ADD CODE HERE: read in an image of a face, call the variable 'img'

# we operate on a grey image, but keep RGB for visualisation later
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

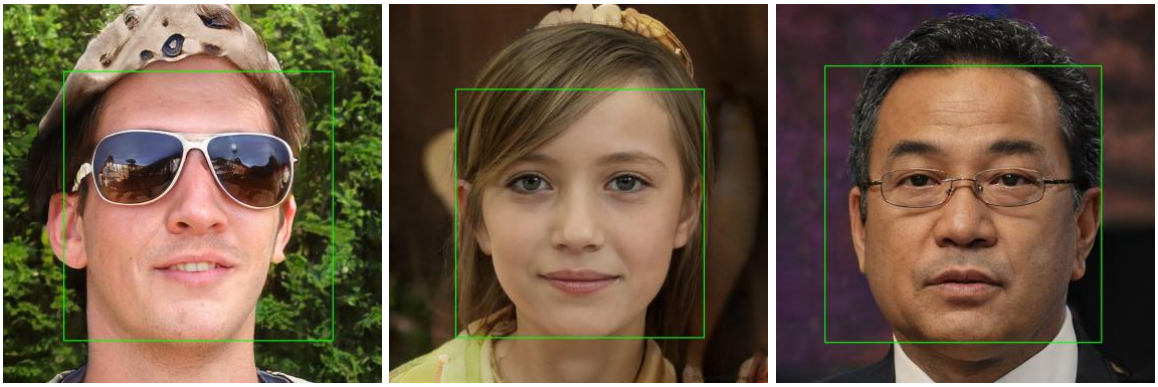
# here we call our two functions, first to get the detections, then
# to draw the resulting rectangles/bounding boxes on the rgb image
rects = detect(gray, cascade)
draw_rects(img, rects, (0, 255, 0))

# show the result
cv.imshow('facedetect', img)
cv.waitKey(0)
cv.destroyAllWindows()
```

Before we can detect faces we need to get two files:

1. *haarcascade_frontalface_default.xml* is available from Blackboard;
2. A face to detect – head to https://www.thispersondoesnotexist.com to get one or more images of faces for testing (note these are auto-generated images, and sometimes you get some unnatural results)

Once you have these, you should see something like this when you run it:



Let's upgrade to video so we can see how this works in realtime. We need to do a bit of restructuring to work with video rather than images, so start a new file and we'll start by getting the video feed from the webcam up and running…

```
import numpy as np
import cv2 as cv

cap = cv.VideoCapture(0) # open the video feed from the webcam
if not cap.isOpened(): # if something goes wrong, exit gracefully
    print("Cannot open camera")
    exit()

while True: # otherwise capture frame-by-frame
    ret, frame = cap.read() # if frame is read OK ret is True
    if not ret:
         print("Can't receive frame (stream end?). Exiting ...")
         break
    # frame is an image, so we can work with it like any other!
    # we could do some processing here, for now just take as-is
    result = frame

    # Display the resulting frame, resize to make it fit on screen
    tmp = cv.resize(result, (0,0), fx=0.4, fy=0.4)
    cv.imshow('webcam', tmp)
    if cv.waitKey(1) == ord('q'):
        break
# When everything done, release the capture
cap.release()
cv.destroyAllWindows()
```

If everything goes well you will get a live webcam feed, and we now need to combine our two pieces of code together:

- Copy your cascade XML and the two functions from the original Viola-Jones code into the start of your webcam file (**after** the imports!)
- Then, replace the line **result = frame** from your video code with the following from our Viola-Jones file:

```
gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY) # make the frame grey
rects = detect(gray, cascade)                # run the detection
result = frame.copy()                        # copy the frame
draw_rects(result, rects, (0, 255, 0)        # draw on the copy
```

You should now be able to get live face detections!

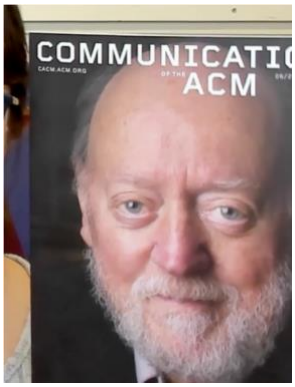
If you want to take the video feed from a file, you replace the 0 with a filename, as in:

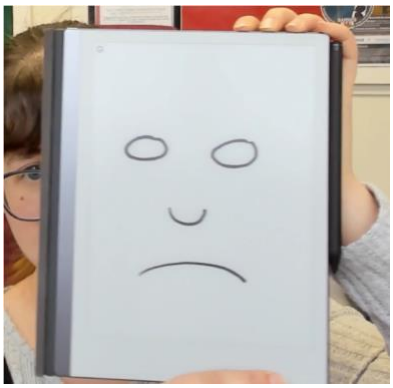
```
cap = cv.VideoCapture(filename)
```

Try loading in the *people.mp4* and *paris.mp4* videos supplied on Blackboard.

But just how good is Viola-Jones at finding faces? Try loading the *face-or-no-face.mp4* video (courtesy of Helen!) to test some possible faces, including…







**A challenge**: what's the simplest face you can draw that Viola-Jones will detect? Can you find a face that's not-a-face but Viola-Jones still detects one?

Think about how you could fool face detection – we will be discussing security applications more in a later lecture.

## Part 2: Finding Objects with a Deep Neural Network

Next, we are going to explore object detection using a Deep Neural Network, which can be done using the **dnn** package within opencv. Detecting and tracking objects can be important in a security setting, such as detecting when people or cars are in a restricted area. We are going to use a fairly lightweight network, that can run fine in the lab machines without needing to use the GPU. Start a new python file, and first set up the loop to capture from

video and display the frames in a loop, as shown earlier. Before the video loop, add code to set up a few variables and load the model:

```
# The path to the files needed for the mobilenet model
proto_file = 'deploy.prototxt'
model_file = 'mobilenet_iter_73000.caffemodel'

# Check they both exist
if not os.path.exists(model_file):
    print(model_file + " does not exist")
    exit()
if not os.path.exists(proto_file):
    print(proto_file + " does not exist")
    exit()

# Load the model!
net = cv.dnn.readNetFromCaffe(proto_file, model_file)

# The required input shape and normalisation for the network
input_shape = (300, 300)
mean = (127.5, 127.5, 127.5)
scale = 0.007843

# Classes in the model
classNames = {0: 'background', 1: 'aeroplane', 2: 'bicycle',
              3: 'bird', 4: 'boat', 5: 'bottle',
              6: 'bus', 7: 'car', 8: 'cat', 9: 'chair',
              10: 'cow', 11: 'diningtable', 12: 'dog',
              13: 'horse', 14: 'motorbike', 15: 'person',
              16: 'pottedplant', 17: 'sheep', 18: 'sofa',
              19: 'train', 20: 'tvmonitor'}
```

Inside the loop run the model on each frame. Remember the `blobFromImage` function from the first practical? Well, we'll use that here to prepare the frame for the network, then pass it to the network, and use the results:

```
#image preprocessing
blob = cv.dnn.blobFromImage(frame, scalefactor=scale,
       size=input_shape, mean=mean, swapRB=True)
net.setInput(blob)
results = net.forward()
```

The results array has the shape `[1,1,N,7]`, where N is the number of detections, and the final dimension gives an array of `[batch, class_id, confidence, x_min, y_min, x_max, y_max]`. Hence, we can loop a variable, e.g. `i`, over the range `results.shape[2]`, and use `results[0,0,i,1]` as the label of the detected object and `results[0,0,i,2]` as the confidence in that prediction. The bounding box is `results[0,0,i,3:7]` are stored in normalised form in the range [0,1], so need to be

multiplied by the width and height of the image to get the bounding box. Putting this together we have:

```
for i in range(results.shape[2]):
      # confidence
      confidence = round(results[0, 0, i, 2],2)
      # class id
      id = int(results[0, 0, i, 1])
      if confidence > 0.5:
            # 3-6 contains the coordinate
            x1, y1, x2, y2 = results[0, 0, i, 3:7]

            # scale these coordinates frame size
            ih, iw, ic = frame.shape
            x1, x2 = int(x1*iw), int(x2*iw)
            y1, y2 = int(y1 * ih), int(y2 * ih)
            cv.rectangle(frame, (x1, y1), (x2, y2), (0, 200, 0), 2)
            cv.putText(frame, f'{classNames[id]}:{confidence*100}',
                        (x1+30, y1-30), cv.FONT_HERSHEY_DUPLEX, 1,
                        (255, 0, 0), 1)
```

Try the object detector out on a few of the videos from Blackboard.

## Link to the assignment

In the assignment you are asked to detect yellow cars. Consider the following questions relating to the assignment, and write down your answers, you will need them for the assignment report.

### CS34020 and CSM4220 Students

Write a short paragraph discussing which of the methods used in this workshop might be suitable to help with the assignment task and which are not, and why. You will be asked to include this in your assignment submission.

### CSM4220 Students only

The CNN used in this workshop is a MobileNet-SSD. By referring to the original mobilenets paper[1], summarize how the network achieves real-time performance on fairly modest hardware.

[1] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H., 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.